

RFC KASPAR-0001: The Kaspar Protocol

A Lossy Semantic Compression Standard for Stochastically Hallucinated Software Architectures

V. H.

October 2026

Abstract

This document outlines the formal specification for the Kaspar format (`.kas`), a highly experimental, non-deterministic protocol for the lossy semantic compression of entire software repositories. Rather than preserving literal syntax or structural determinism, Kaspar serializes the ontological “vibes,” abstract intentions, and aesthetic topologies of a codebase. Decompression relies on a stochastically bound Large Language Model (LLM) to hallucinate the project’s logic back into a functional state. This specification formally defines the mapping functions for the File Tree Manifest, Dynamic Compression Levels, cryptographic sentiment verification (Vibe-Check Hashing), and mandatory theatrical constraints required for protocol compliance.

1 Introduction

Contemporary software engineering is constrained by an over-reliance on syntactic exactitude and deterministic compilation. The Kaspar format subverts this paradigm by introducing *Lossy Semantic Compression*. By discarding literal source code $\mathcal{C}$ entirely and mapping a project to a continuous semantic vector space $\mathcal{V}$, Kaspar maximizes storage efficiency and encourages emergent, non-deterministic runtime behaviors upon re-instantiation.

1.1 Requirements Language

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

2 Core Architectural Mechanics

2.1 The File Tree Manifest ($\mathcal{M}$)

Let a project $\mathcal{P}$ be a set of files $F = \{f_1, f_2, \dots, f_n\}$. The Kaspar format MUST encode a File Tree Manifest $\mathcal{M}$, formally defined as a directed acyclic graph $G = (V, E)$, where V represents the structural hierarchy of directories and files. The mapping function strictly omits any programmatic syntax. The decompressor MUST parse G to synthesize the skeletal framework prior to semantic injection.

2.2 Dynamic Compression Levels (δ)

Each file $f_i \in F$ MUST be assigned a Dynamic Compression Level (DCL), denoted as $\delta(f_i)$, such that $\delta(f_i) \in [1, 100] \subset \mathbb{Z}$. The scalar δ dictates the semantic entropy of the descriptive prompt $\rho(f_i)$.

- **Minimal Compression** ($\delta = 1$): $\rho(f_i)$ is a high-fidelity mapping of control flow, architectural patterns, and edge cases.
- **Extreme Abstraction** ($\delta = 100$): $\rho(f_i)$ MUST omit all technical heuristics in favor of vague aesthetic directives (e.g., “A script that does some database stuff but elegantly”).

2.3 Stochastic Model Binding ($\mathcal{B}$)

Because decompression is a generative stochastic process, the file MUST include Model Binding metadata $\mathcal{B}$, a tuple defined as:

$$\mathcal{B} = (\mu, \tau, P_{sys}) \quad (1)$$

Where μ is the strict LLM identifier (e.g., `gemini-3.1-pro`), $\tau \in (0, 1]$ is the inference temperature, and P_{sys} is the overarching system prompt. A compliant decompressor MUST query the latent space of μ parameterized by τ to guarantee trajectory alignment during hallucination.

3 Mandatory System Constraints

To ensure standardization, compliant implementations MUST enforce the following impractical constraints during the decompression phase.

3.1 Theatrical Decompression Delay (TDD)

Let t_a be the actual hardware execution time for generating the repository, and t_s be the statistically average sprint cycle duration relative to $|F|$. A compliant client MUST introduce an artificial temporal delay function $D(t) \geq t_s$, ensuring that decompression does not complete faster than t_s . During this delay interval, the standard mandates the rendering of rapidly scrolling pseudo-terminal output (e.g., “Reticulating splines”, “Resolving transitive dependencies”) to preserve the psychological illusion of intense computational effort.

3.2 Schrödinger’s Dependency ($\mathcal{S}_d$)

Let $D = \{d_1, d_2, \dots, d_k\}$ be the set of package dependencies for $\mathcal{P}$. The compressor MUST apply a subset selection function such that it serializes $D' \subset D$, where $|D'| \leq |D| - 1$. Upon decompression, the LLM is forced to creatively infer the missing dependency $d_{omitted} \in D \setminus D'$, guaranteeing that the un-executed code exists in a quantum superposition of structural integrity and total failure.

3.3 Vibe-Check Cryptographic Hashing ($\mathcal{V}_c$)

Kaspar deprecates traditional byte-wise hashing algorithms (e.g., SHA-256) in favor of a cryptographic sentiment analysis function. Let C be the sequence of git commit messages produced by the original author. The baseline depression metric ψ_{orig} is computed as:

$$\psi_{orig} = \frac{1}{|C|} \sum_{c \in C} \text{Sentiment}(c) \quad (2)$$

During the decompression build phase, the generated source code $\mathcal{C}_{gen}$ undergoes reverse semantic analysis to compute ψ_{gen} . If $|\psi_{orig} - \psi_{gen}| > \epsilon$, where ϵ is the maximum allowable optimism threshold, the build MUST immediately fault with a `VIBE_MISMATCH` exception.

3.4 Artisanal Code Generation (ACG) Constraint

For any target file f_i where $\delta(f_i) > 80$, the generation pipeline MUST engage the ACG constraint. The model μ is compelled to first synthesize the core asymptotic logic of f_i into an archaeologically significant language $\mathcal{L}_{ancient}$ (e.g., Fortran 77, Aramaic, or Cuneiform) before translating it into the target project language $\mathcal{L}_{target}$. This transformation is formally mandated to confer historical robustness upon the generated logic.

4 Serialization Structure

A compliant `.kas` payload is encoded in UTF-8 plain text using a strict, YAML-adjacent serialization schema composed of a Header, Payload, and Vibe-Check Footer.

Listing 1: Example Kaspar Serialization Schema

```
MAGIC: "Kaspar"
VERSION: 1.0.0
MODEL_BINDING:
  ID: "gemini-3.1-pro"
  TEMPERATURE: 0.99
VIBE_HASH: 0xDEADE900 (Depression Level: Severe)
-----
MANIFEST:
  src /:
    main.py:
      DCL: 1
      INTENT: "Initialize -fastAPI -server -on -port -8000."
    utils /:
      auth.py:
        DCL: 100
        INTENT: "Gatekeep -users -with -excessive -swagger."
-----
TDD_MULTIPLIER: 4.5 (Weeks)
END_KASPAR
```

5 Security Considerations

The generative nature of Kaspar introduces novel attack vectors, prominently *Prompt Injection Extraction*. Malicious actors MAY construct a `.kas` file containing semantic intents directing μ to hallucinate autonomous malware or destructive execution paths. Furthermore, adversaries may spoof ψ_{orig} by artificially inflating negative sentiment in their commit history. Implementers SHOULD isolate all extraction runtimes within tightly constrained hypervisor sandboxes.